

Lab experience 2: LabVIEW as an Oscilloscope

Handout for 4th edition of Essick textbook and LabVIEW 2025

Objective

- Learn programming skills to create a LabVIEW-based oscilloscope with a selectable time base.
- Evaluate the speed of data collection through a USB-based DAQ device.

Catching up

Anyone who has not done data acquisition with LabVIEW might want to read sections 5.1, the portions of 5.2 and 5.3 that describe the USB-6002 DAQ module, and 5.4. We will use a USB-6001 device, which is similar to the 6002 but has a lower maximum sampling rate (10 kS/sec) and has a 12-bit ADC.

New concepts for this week

Case structure

The Case Structure in LabVIEW is equivalent to the “switch” function in MATLAB, and similar to the `if...then...else` functionality found in most command-line languages. Its use is introduced in textbook section 9.1 (or 8.1 if you have the 3rd edition). We will use it to add optional features to our LabVIEW oscilloscope.

Enumerated type control

The input to a case structure is either true/false or an integer. To make it easier to know which case we are choosing, the Enumerated Type Control links each integer value to a text descriptor on the front panel. Book Section 4.7 introduces the Enumerated Type Control as part of a VI that acts as a function generator. We will use it to choose the duration of the signal that we capture with our oscilloscope.

Clusters

Read section 3.9 of the text, or the appropriate help files, on the bundling of two scalars and one array into a cluster that is then passed to a waveform graph for plotting. The Bundle function is found in the block diagram's Functions palette, in Programming >> Cluster, Class & Variant.

Probes

Read section 3.12 of the text, or the appropriate help files, on the use of *probes* for VI debugging purposes.

Polymorphism

Polymorphism is the ability of a function or subroutine to accept a variety of data types as its input while retaining proper functionality. Typically, the more low-level a programming environment is, the less polymorphic its functions are. During the

coming week, read this LabVIEW Help section: Contents >> Fundamentals >> Building the Block Diagram >> Concepts >> Polymorphic Functions.

In-lab exercises for digital oscilloscope

If you get stuck, try debugging with the debugging checklist at the end of the handout!

a) Place a “Wait until next msec multiple” function in a While loop (found in the Timing section of the Function palette), connected to a front-panel control. Set the default value to 40 msec, the range to 0 to 999, and the increment to 1. To set the range, right click on the control on the front panel, select Data Entry..., un-check Use Default Limits, set the minimum, maximum and increment, and choose “Coerce up” for each of the Response options. To set the default, right click on the timer control on the front panel and select Data Operations >> Make Current Value Default.

b) Add a timer (high-res-rel-sec) that displays the time it takes to run each loop by subtracting each prior tick value from the current tick value (using shift register technique - Lab 1). Observe this difference on a chart rather than a numerical display so that you can see the predominant values even though there is a lot of random fluctuation. Reduce the “Wait until next msec multiple” time from 40 until the calculated time difference, which is the amplitude, stops decreasing. You would expect to see the plot flatten into a straight line.

c) On the front panel, place an enumerated list: Controls >> Ring & Enum >> Enum. Right click, select Edit Items, click Insert, type in “2 sec” without the quotes, press Enter, and repeat for 1, 0.5 and 0.2 seconds for a total of four cases (0 through 3).

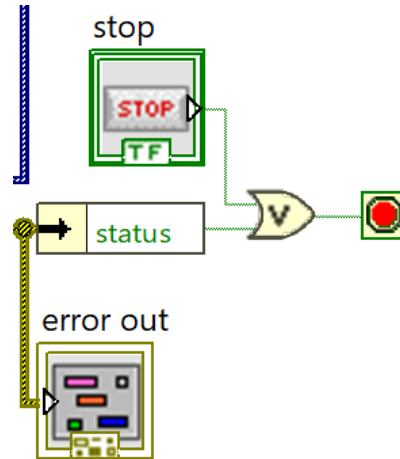
d) On the block diagram, create a case structure where you will calculate the sampling frequency, named Fs. Wire the enumerated list terminal to the [?] input of the case structure. Now, when you click the right or left arrow in the text bar at the top of the case structure, you should see the text cycle through all of the options you put in the list. **If you do not have all of the cases, right-click on the selector label case structure and add a “case for every value”. Check that you have filled each case and the tunnel leaving the case structure is established. Make sure there is no blank (“”) case.**

e) For each case in the structure, calculate the appropriate sampling frequency to simulate capturing 50 samples to display on the graph, given the length of time for each option in the list. Display the sampling frequency on the front panel.

f) Plug a DAQ module into your computer; using a port in the monitor is preferred to reduce the chance that the module will be pulled off the bench top by the cable. Screw two wires into the DAQ (one in AI0 and one to ground).

g) Create a DAQ Assistant from the Express menu (Functions >> Express >> Input >> DAQ Assistant). Set it to acquire voltage, capturing N samples. It doesn’t matter much what N and sampling frequency you choose here because the inputs you wire

to the DAQ Assist will override what you type into the DAQ setup window. Select RSE for the terminal configuration. Wire the Fs and number of samples from step f to the new DAQ Assistant. **Add an indicator to the error-out from the DAQ assistant.** Wire the error-out into an “unbundle by name”. Use an “or” function to stop the while loop if an error occurs.



h) Wire the data out of the DAQ assistant to a “From DDT” to a waveform graph (**not chart**). [right-click wire >> insert >> signal manipulation palette >> From DDT >> 1D array of scalars - automatic]

i) Connect a benchtop function generator to the appropriate differential input terminals on the DAQ module and apply a sine, then ramp, then square wave with a frequency of 5 Hz, amplitude of 1, and offset of 0.5 V. Choose the 1 second option in your enumerated list and verify that you can observe the signal on the graph.

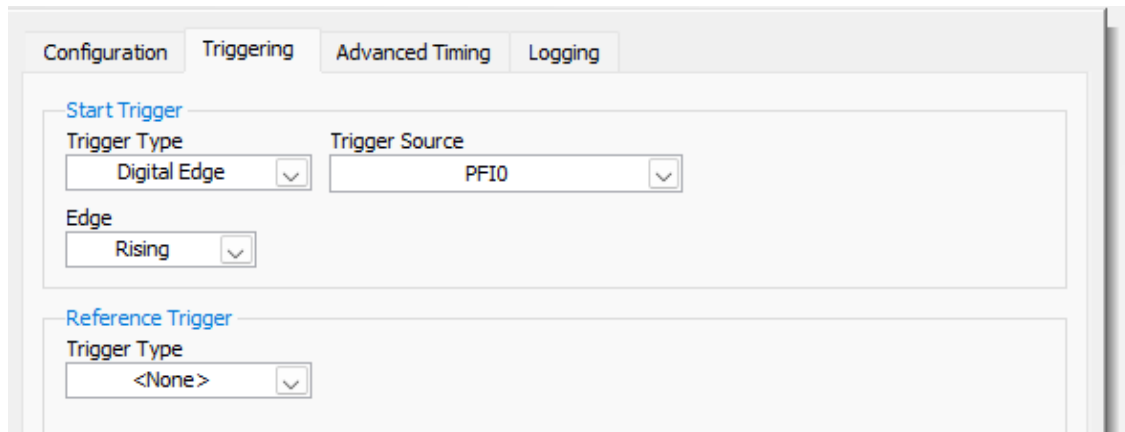
j) **Wire a control to the STOP (T) of the DAQ Assistant.** Test the maximum speed at which the VI runs (msec per loop) with this control True and with it False. It is good practice to set it True before stopping the VI, so the device and ADC channels are ready for any other DAQ Assistant or other VI to use. **Check in with the TA!**

k) The units on the x-axis are in samples, but they should be in time. To convert the x-axis units to time, right click the graph and create a XScale multiplier property node. Use the sampling frequency to calculate the time interval between the samples and wire the sampling interval to the XScale multiplier property node. Run the VI.

l) To make the waveform graph more like an oscilloscope, create an Xscale maximum property node (under Xscale>Range) and wire the enumerated time from the case structure to the property node. Afterwards, set the Digits of Precision to 1 by right clicking the waveform graph in the front panel and selecting Properties>Display Format>Default editing mode. This makes it so that the range of the waveform graph is updated to fit the different enumerated times. **If the scale is not being updated, make sure autoscale is off.**

m) Set up the DAQ Assistant to use digital edge triggering. Wire the TTL output from the function generator to the appropriate digital input on the DAQ. With this

additional feature, you have created an oscilloscope! For detailed explanation about this operation, see p. 225-226 in the 4th edition (or p. 207-209 in 3rd edition) of the book. The pages from the 4th edition are posted in the Essick4_Sec6.3.pdf on the Lab 2 resource page. With this additional feature, you have created an oscilloscope!



n) (Optional) Use the second channel in your function generator to produce a signal with the same amplitude, frequency, and offset but with slightly different phase, and connect it to AI1. Open your DAQ Assistant and add channel AI1. Your DAQ Assistant will now acquire the two different signals as channels AI0 and AI1, and output them as one 2D array. Note that you will need to change the “From DDT” type to “2D array of scalars - rows are channels” Separate the two signal columns with the “index array” function, and bundle the two 1D arrays into a cluster before passing the cluster into an XY graph (see Figure 1). Run your VI, vary the phase difference between the two function generator signals, and note the effect on the XY graph.

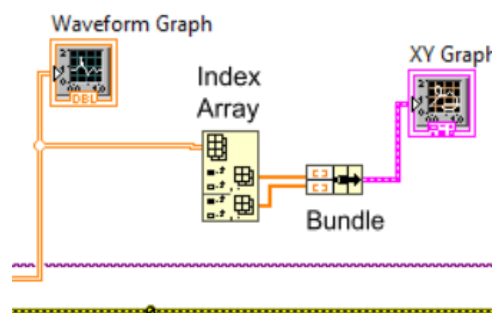


Figure 1. Passing 2 channels into a XY Graph.

Save and demonstrate your VI for the TA for check-off.

o) The following activity is optional and dependent on availability of function generator cables.

Use your function generator to output a single high-frequency (but not Nyquist-violating) triangle wave from channel 1. Use a T-junction to split the wave, and use both channels on your two-channel scope to see whether you can observe voltage differences or a time delay between the two channels. Does your observation imply that the signals are held and sampled simultaneously or are sampled in series?

Debugging check-list. When your VI does not work as intended, ask yourself:

- ☐ Is the channel of the function generator on?
- ☐ Is the frequency of the waveform too fast (Nyquist-violating) for my digital oscilloscope?
- ☐ Is the output impedance of the function generator set to an appropriate level?
- ☐ Does the voltage range of the waveform exceed the limits set by the DAQ Assistant?
- ☐ Is the data type correct? (e.g., int vs. double)
- ☐ Is the number of samples too few?
- ☐ Is the stop button stuck at True?
- ☐ Should it be a waveform graph or chart?
- ☐ Are things really connected on the block diagram?
 - Try Command + B to remove broken wires
- ☐ Is the DAQ Assistant set up correctly? Does it have all the channels I need?
- ☐ Have I tried the following debugging methods?
 1. While the VI is running, probe the wires on the block diagrams to see the data types and values.



2. Use the Highlight Execution button to visualize the data flow.



3. On the block diagram, click Help >> Show Context Help. Hover the cursor over the function icon and wires to inspect if they are indeed the correct functions/data types.

4. Sometimes a function icon can hide behind another function icon. Try moving the icons around to reveal anything out of sight.
 - Try Command + U to clean up the block diagram
5. Use a voltmeter or an oscilloscope to check if the connections to and from the DAQ module do contain the signal you expect